

Remark 2.6. Under our assumptions, the distribution of the LM’s prediction of the next token may not necessarily converge to a fixed distribution, since $\frac{\sum_{j=1}^n \alpha_i[j]}{\sum_{j=1}^n \alpha[j]}$ is not a constant but rather a function of n . Specifically, $\hat{p}_n(v_i|\mathbf{x})$ may continue to vary with increasing n , regardless of how large n becomes. Nonetheless, **we prove that when n is sufficiently large, the output distribution of generation n does not necessarily approximate the ground-truth distribution of the initial corpus—a property that holds irrespective of whether the output distribution converges to any particular distribution. Section 3.2 depicts this property.**

Under the *Accumulate-Subsample* paradigm and the same assumptions, the series $k \sum_{j=1}^{n-1} \frac{1}{1+jk} \alpha[j]$ also diverges. This leads to the following conclusion in the Main Theorem:

$$\forall \epsilon > 0 \exists n_1 \in \mathbb{N} \forall n > n_1 \left(\left| \hat{p}_n(v_i|\mathbf{x}) - \frac{k \sum_{j=1}^{n-1} \frac{1}{1+jk} \alpha_i[j]}{k \sum_{j=1}^{n-1} \frac{1}{1+jk} \alpha[j]} \right| < \epsilon \right).$$

The following section will provide some final discussion regarding this conclusion for the *Accumulate-Subsample* paradigm.

2.7 Conclusion on the existence and implications of LM collapse

Section 2.1.1 defines k as the rate at which the generated data enters the corpus. Note that the magnitude of k does not affect our proof. This property indicates that once the corpus begins to expand due to generated data, while the amount of training data for each generation of LMs remains unchanged, LM collapse will eventually occur. It is just a matter of time. In practice, if the corpus is large and k is small, we can be relatively sure that the collapse of LM will not occur in the short term. However, by proving the *existence* of LM collapse, we reasonably express our concerns about the current situation where an increasing amount of generated data is used in LM training.

3 Experiments

We empirically analyze the output distribution of each LM generation to demonstrate our theoretical findings.

3.1 Experimental setup

Following the *Replace* data paradigm defined in Section 2.1.1, we train generation n exclusively on the data generated by generation $n - 1$. As demonstrated by our Main Theorem, the outcomes of LM collapse exhibit no fundamental distinction between the *Replace* and *Accumulate-Subsample* paradigms as n approaches infinity. Since LM collapse may take many generations, using *Replace* allows for earlier observation of experimental phenomena.

3.1.1 Training corpora

For generation 1, we select a popular corpus named *TinyStories* (Eldan & Li, 2023) as its training corpus. *TinyStories* consists of 2.12×10^6 short English stories. The stories contain only words that most 3 to 4 year olds would typically understand. They are so simple that they can be used to train and evaluate small language models (SLMs) that are much smaller than the state-of-the-art models, yet still enable them to produce a diverse set of fluent and consistent stories. The training of SLMs on *TinyStories* can typically be done in less than a day on a single GPU from scratch. Therefore, corpora like *TinyStories* facilitate our experiments.

For generation n ($n \geq 2$), we use generation $n - 1$ to generate stories as the training corpus. Each story is generated freely, *i.e.*, starting from an empty prompt. The tokens of the story are repeatedly sampled from the output distribution of the LM. The scale of the generated corpus is the same as *TinyStories*. Details can be found in Appendix C.1, and Table 1 in which segment of the corpora generated by LMs of various generations are shown, demonstrating what these corpora look like.

3.1.2 Language models

Our LMs utilize the classic GPT-Neo architecture (Black et al., 2021). We train two series of LMs. In one series, all LMs have 1 million (1M) parameters. In the other series, all LMs have 33 million (33M)

parameters. For detailed architectures, please refer to Appendix C.2. We train the 1M LMs on the text generated by the previous 1M LMs and the 33M LMs on the text generated by the previous 33M LMs. One can inspect if LMs with more parameters are more resilient to performance degradation during recursive training.

3.1.3 Inspections carried out on each generation of LMs

Output distribution of generation n on fixed prompt (Figure 2) We demonstrate the numerical values of $\hat{p}_n(v_i|x)$ in our theoretical part by inputting the same input x into each generation of LMs.

Corpus-level divergence of generation n (Figure 3) We compare the ‘average distance’ between the output distributions of each generation and the distributions of the initial corpus using model perplexity (test loss) on the validation set of TinyStories. The formula for calculating this distance can be found in Appendix C.3.

3.2 Experimental results

Figure 2 demonstrates the evolution of $\hat{p}_n(v_i|x)$ values for n ranging from 1 to 40, where the token sequence x is a single token ‘there’ and v_i represents either ‘was’ or ‘were’. This case study was specifically selected for its clear demonstration of LM prediction divergence from the ground-truth distribution across generations. In the initial TinyStories corpus, we observe $p(\text{was}|\text{there}) = 0.76$ and $p(\text{were}|\text{there}) = 0.05$, represented by horizontal reference lines of True $p(\text{was}|\text{there})$ and True $p(\text{were}|\text{there})$ in the figure.⁵

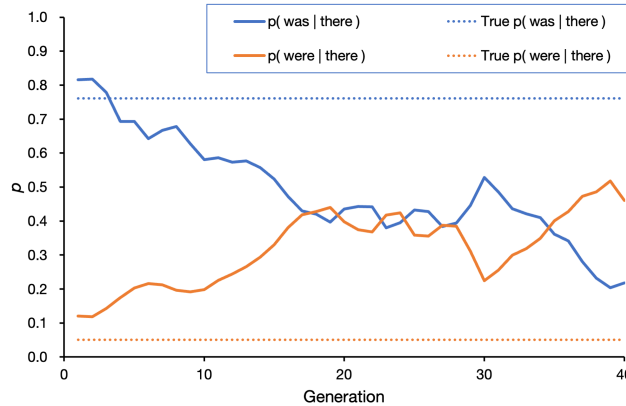


Figure 2: Language model (LM) collapse is a degenerative process whereby, over generations, LMs forget the underlying text distribution of the initial training corpus. Our experiment begins with an initial corpus used to train the LM at generation 1. Then, generation 2 is trained using the text generated by generation 1, generation 3 using the text generated by generation 2, and so on. The figure depicts the process of LM collapse. The dotted lines True $p(\text{was}|\text{there})$ and True $p(\text{were}|\text{there})$ refer to the probabilities of two phrases ‘there was’ and ‘there were’ given ‘there’ in the initial corpus TinyStories, which are 0.76 and 0.05 respectively. The first-generation LM with 33 million parameters trained on it can learn this probability well (the solid lines $p(\text{was}|\text{there})$ and $p(\text{were}|\text{there})$ represent the probabilities that the LMs predicts the next token after ‘there’ to be ‘was’ and ‘were’ respectively). However, as n increases, the solid lines gradually deviate from the dotted lines, underscoring the growing disconnect between the output of LMs and the initial training text.

Our analysis reveals that LM outputs initially approximate the corpus distribution well, with $p_n(\text{was}|\text{there})$ and $p_n(\text{were}|\text{there})$ closely matching their respective ground-truth probabilities when n is small, indicating that our LMs are able to learn this distribution in the initial corpus. However, as n increases, the probabilities output by LMs deviate significantly from those in the initial corpus, which corroborates the result of our theoretical proof. Note that this deviation does not increase

⁵Most of the stories are in the past tense, so the probabilities of ‘there was’ and ‘there were’ occurring are relatively high, while the probabilities of ‘there is’ and ‘there are’ occurring are relatively low.

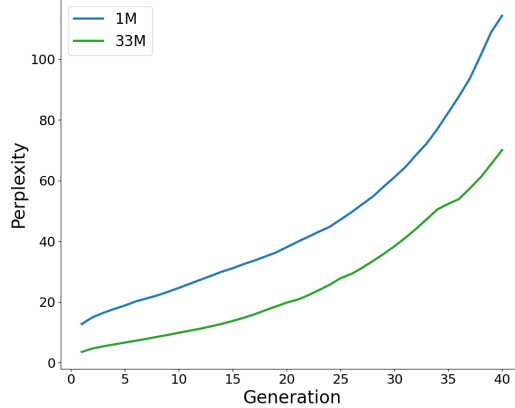


Figure 3: **Perplexity of LMs with 1M and 33M parameters over 40 generations evaluated on the validation set of the TinyStories dataset.** Model perplexity is not only an indicator of the performance of LMs, but also a measure of the ‘distance’ between the distributions learned by the LM and the distributions of the initial corpus. (The formula for calculating perplexity can be found in Appendix C.3.) The increase in perplexity is due to the increase in this distance, which means that the learned distribution is becoming increasingly deviated from the initial corpus. This corroborates our theoretical result.

monotonically with n . For example, the difference between $p_n(\text{was}|\text{there})$ and $p(\text{was}|\text{there})$ temporarily decreases in some generations. It does not violate our theory because even if the probability fluctuates as n increases and continues to fluctuate forever, we have proven that the LM’s prediction of the next token ceases to accurately reflect the original real-world data distribution. Moreover, from a statistical perspective, the perplexity (equivalent to test loss) of generation n on the initial corpus can be used to measure the degree to which its output distribution deviates from the ground-truth distribution on average. As shown in Figure 3, this metric reveals a clear trend of increasing deviation when local fluctuations from individual token examples are aggregated, providing stronger evidence of LM collapse than single-token trajectories. We make available our code for reproducing our results as well as probing output probabilities with custom prompts $\mathbf{x}$ in the supplementary material.

4 Discussion

Our theoretical proof establishes that LM collapse occurs regardless of the rate k at which generated data infiltrates the initial real-world dataset, as long as no new real-world data is introduced. This finding suggests that attempts to mitigate collapse by limiting the *quantity* of synthetic data in the corpus are fundamentally insufficient. Instead, avoiding collapse hinges on ensuring the *quality* of synthetic data.

This conclusion aligns with contemporaneous works exploring collapse prevention. For example, work by Zhu et al. (2024) demonstrates that token-level editing techniques can effectively counteract collapse by refining synthetic data quality, even when models are trained recursively on edited outputs. Such approaches validate our theoretical insight: collapse arises not from the mere presence of synthetic data, but from its progressive divergence from the ground-truth data distribution across generations. By theoretically demonstrating that *any* amount of uncured synthetic data accumulation leads to eventual collapse, our work clarifies the futility of strategies focused solely on regulating the *proportion* of generated data.

These findings call for research attention towards a critical direction: developing robust methods to assess and enhance the quality of synthetic data at scale. Promising avenues include (1) refining data filtering mechanisms to retain high-entropy, diverse patterns in generated text, (2) integrating human or automated feedback loops to improve synthetic data iteratively, and (3) hybrid training paradigms that strategically blend real and high-quality synthetic data. Ultimately, our proof underscores that the sustainability of LM training hinges not on avoiding synthetic data entirely, but on rigorously ensuring its fidelity to the original data distribution.